



Sinfama

Caja de compensacion

Agente SQL - Procesos RPA

Guia de Instalacion y Ejecucion

Este documento describe como clonar, configurar y ejecutar el proyecto de analisis de procesos RPA con agente SQL local (Ollama) y prediccion de ROI (XGBoost + GPU) en cualquier equipo con Windows, macOS o Linux.

1. Requisitos previos

Herramienta	Version minima	Descarga
Python	3.11	https://python.org/downloads
Git	cualquier	https://git-scm.com
Git LFS	cualquier	https://git-lfs.github.com
Ollama	0.23+	https://ollama.com/download
CUDA Toolkit	12.x (op)	https://developer.nvidia.com/cuda-downloads

Nota: GPU NVIDIA es opcional. XGBoost detecta CUDA y hace fallback a CPU automaticamente.

2. Instalar Git LFS (obligatorio para las bases de datos)

El repositorio usa Git LFS para los archivos .db (bases de datos SQLite, ~300 MB). Sin esto los archivos quedaran vacios.

```
# Windows (winget)
winget install GitHub.GitLFS

# macOS
brew install git-lfs

# Ubuntu / Debian
sudo apt install git-lfs

# Activar globalmente (una sola vez por maquina)
git lfs install
```

3. Clonar el repositorio

```
git clone https://github.com/miloguz/Proyecto1Especializacion.git
cd Proyecto1Especializacion

# Si los .db quedaron vacios:
git lfs pull
```

4. Instalar dependencias Python

Opcion A - con uv (recomendado, resuelve el entorno automaticamente):

```
pip install uv
uv sync
```

Opcion B - con pip estandar:

```
pip install streamlit ollama plotly xgboost joblib pandas \
    scikit-learn seaborn matplotlib numpy
```

5. Instalar Ollama y descargar el modelo LLM

El agente SQL necesita Ollama corriendo localmente y el modelo qwen2.5-coder:7b (~4.7 GB).

```
# Terminal 1 - servidor Ollama
ollama serve

# Terminal 2 - descargar modelo (solo la primera vez)
ollama pull qwen2.5-coder:7b
```

Nota: En Windows, Ollama se instala como servicio y arranca automaticamente. No es necesario 'ollama serve' de forma manual.

6. Entrenar el modelo de prediccion de ROI

El archivo models/roi_model.joblib NO esta en el repositorio (figura en .gitignore).

Hay que generarlo una vez antes de usar la pestana 'Prediccion ROI':

```
python -c "
import sys; sys.path.insert(0, '.')
from src.utils.roi_calculator import build_roi_dataset
from src.models.roi_predictor import train
metrics = train(build_roi_dataset())
print('Dispositivo:', metrics['device'].upper())
print('Modelo guardado en models/roi_model.joblib')
"
```

Nota: Con GPU NVIDIA y CUDA instalado, el entrenamiento usa la GPU automaticamente.

7. Ejecutar la aplicacion

```
# Opcion A - script rapido (solo Windows)
run_app.bat

# Opcion B - manual (Windows / macOS / Linux)
streamlit run app/chat.py
```

8. Solucion de problemas comunes

Sintoma	Causa probable	Solucion
Bases de datos vacias (0 bytes)	Git LFS no instalado	git lfs install && git lfs pull
ModuleNotFoundError: streamlit	Deps no instaladas	pip install streamlit o uv sync
'Ollama no esta corriendo'	Servidor Ollama apagado	ollama serve en terminal aparte
Prediccion ROI falla al cargar	Modelo no entrenado	Ejecutar el paso 6
XGBoost no detecta GPU	CUDA Toolkit no instalado	Instalar CUDA; fallback CPU automatico

9. Estructura del proyecto

```

Proyecto1Especializacion/
  app/chat.py          <- streamlit run app/chat.py
  assets/logo.svg     <- logo Sinfama
  src/
    agent/             <- agente SQL + conexion DB
    models/roi_predictor.py <- XGBoost GPU/CPU
    utils/roi_calculator.py <- calculo de ROI
  data/database/
    Procesos_clean.db  <- base de datos (Git LFS)
  models/              <- roi_model.joblib (generado local)
  notebooks/           <- EDA y entrenamiento paso a paso
  run_app.bat          <- acceso rapido Windows
  pyproject.toml       <- dependencias del proyecto

```

10. Formula de calculo de ROI

```

Beneficio_Bruto = TiempoManual x ValorHora x NumEjecuciones
Costo_Robot     = DuracionRobot x 7300 x NumEjecuciones
Ahorro_Neto     = Beneficio_Bruto - Costo_Robot
ROI%            = (Ahorro_Neto / Costo_Robot) x 100

```

El costo operativo del robot se estandariza en 7.300 COP/hora para todas las soluciones del portafolio. Este valor proviene de la infraestructura real:

- Servidor Azure: 150.000 COP/mes
- Licencia UiPath robot + orquestador: 5.200.000 COP/mes
- Total mensual: 5.350.000 COP / ~730 h ~= 7.300 COP/h

Aplicar una tarifa uniforme hace comparables los ROIs entre proyectos sin importar el rol que cada bot reemplace.

El modelo predictivo (XGBoost) aplica transformacion log1p(ROI) antes de entrenar para manejar la distribucion sesgada del target.